MAINTENANCE GUIDE

Guía de Mantenimiento

Emulador VASSAL para OCS en iPad

Repositorio: github.com/JaviARQ/vassal-ocs

Versión: 1.0 · Noviembre 2026

Sistema: Windows + GitHub Pages + iPadOS Safari

Índice

1. Introducción y contexto del proyecto.....

1. Introducción y contexto del proyecto.....1

1.1 Estructura del proyecto local.....1

1.2 URLs y datos clave.....4

2. Sincronización con el repositorio original.....4

2.1 Concepto: origin vs upstream.....5

2.2 Comprobación del remote upstream.....5

2.3 Rutina de sincronización.....6

2.4 Resolución de conflictos.....8

2.5 Cuándo y cómo upstream nuestro parche Windows.....11

3. Añadir juegos nuevos al proyecto.....13

3.1 Consideraciones previas.....14

3.2 Paso 1 — Descarga del .vmod.....14

3.3 Paso 2 — Colocación en el proyecto.....15

3.4 Paso 3 — Generación del índice de presets.....16

3.5 Paso 4 — Prueba en local (dev server).....17

3.6 Paso 5 — Commit y push.....18

3.7 Paso 6 — Verificación del deploy.....19

3.8 Lista orientativa de módulos OCS.....19

4. Actualización del iPad para juego offline.....20

4.1 Concepto del service worker.....20

4.2 Procedimiento por módulo nuevo.....21

4.3 Verificación de caché.....21

4.4 Cómo forzar refresco si la app no se actualiza.....22

4.5 Cómo resetear la caché del SW por completo.....23

En el PC (Chrome/Edge).....23

En el iPad.....24

5. Otras actividades de mantenimiento....25
5.1 Verificación del workflow GitHub Actions....25
5.2 Renovación del Personal Access Token (PAT)....25
Renovación....26
Actualizar en Windows....26
5.3 Monitoreo del tamaño del repo....27
5.4 Bump del cache version (cuándo y por qué)....29
5.5 Backup del archivo presets.json....31
5.6 Limpieza del .gitignore....31
5.7 Reportar bugs al autor original....33
5.8 Probar nuevos módulos antes de subirlos....34
6. Troubleshooting....35
6.1 Errores de git....35
6.2 Errores de npm....36
6.3 Errores del workflow GitHub Actions....36
6.4 Errores de carga del módulo en el emulador....36
6.5 Errores en el iPad....36
7. Cheat sheet — comandos rápidos....36
7.1 Setup inicial (una sola vez)....36
7.2 Rutinas habituales....37
7.3 URLs importantes....37
7.4 Archivos clave del proyecto....37
8. Glosario....37
8.1 Git y GitHub....38
8.2 Node.js, npm y Vite....38
8.3 VASSAL y OCS....38
8.4 Service worker y PWA....38
8.5 GitHub Pages y GitHub Actions....38

Nota: este índice se genera mediante campos. Para actualizar los números de página tras editar, haz clic derecho sobre el índice y selecciona "Actualizar campo".

1. Introducción y contexto del proyecto

Este documento recoge las rutinas de mantenimiento del proyecto vassal-ocs, una bifurcación del emulador TypeScript de VASSAL publicada por TyroneSlothrop en GitHub. El proyecto está desplegado en GitHub Pages y permite jugar módulos VASSAL —en particular los de la serie OCS (Operational Combat Series)— offline desde un iPad sin necesidad de Java ni de servidor dedicado.

La guía se redacta como referencia viva: está pensada para consultarse cada vez que se quiera actualizar el código desde el repositorio original, añadir un módulo nuevo, mantener el cache offline del iPad o diagnosticar un problema. Cada sección incluye los comandos exactos a ejecutar en una ventana de símbolo del sistema (cmd) de Windows, los resultados esperados y los errores típicos con su solución.

1.1 Estructura del proyecto local

El proyecto se encuentra en C:\Users\JaviPC\vassal-ocs. La estructura interna relevante para el mantenimiento es la siguiente:

```
vassal-ocs/
├── .github/workflows/
│   └── deploy.yml           ← workflow de GitHub Actions
├── .gitignore               ← creado por nosotros (excluye
node_modules, dist, etc.)
├── index.html               ← registra el service worker
├── package.json             ← dependencias y scripts npm
├── package-lock.json
├── vite.config.ts           ← base: "./" para servir en
subdirectorio
├── tsconfig.json
├── scripts/
│   └── gen-presets.mjs      ← PARCHEADO: usa fileURLToPath (fix
Windows)
├── src/                     ← código TypeScript del emulador
├── public/
│   ├── sw.js                ← service worker (offline)
│   ├── sw-register.js       ← registro del SW desde index.html
│   └── modules/
│       ├── presets.json     ← generado por npm run presets
│       └── tunisia-ii/
│           └── Tunisia_II_1_17_2.vmod
└── dist/                    ← generada por npm run build (no se
commitea)
```

La carpeta node_modules y la carpeta dist no se suben a GitHub: la primera se reconstruye con npm install, la segunda se genera en el workflow de Actions.

1.2 URLs y datos clave

Concepto	Valor
Repositorio propio (origin)	https://github.com/JaviARQ/vassal-ocs
Sitio publicado (Pages)	https://javiarq.github.io/vassal-ocs/
Repositorio original (upstream)	https://github.com/TyroneSlothrop/vassal-ts-emulator
Ruta local del proyecto	C:\Users\JaviPC\vassal-ocs
Usuario GitHub	JaviARQ
Branch principal	main
Workflow de deploy	.github/workflows/deploy.yml
Service worker	public/sw.js (cache: vassal-ocs-v1)

2. Sincronización con el repositorio original

El emulador lo mantiene activamente TyroneSlothrop. Cuando el autor publica mejoras (nuevos traits soportados, corrección de bugs, optimizaciones para iOS), interesa absorber esos cambios en nuestra bifurcación sin perder los añadidos propios: módulos cargados, .gitignore, parche Windows del gen-presets.mjs y service worker.

2.1 Concepto: origin vs upstream

Git permite tener varios remotes (punteros a repositorios remotos). Por defecto, al clonar un repo se crea el remote origin apuntando al repo del que se clonó. Para seguir la evolución del original, se añade un segundo remote llamado upstream.

Remote	URL	Para qué sirve
origin	https://github.com/JaviARQ/vassal-ocs.git	Es TU repo. Aquí empujas tus cambios.
upstream	https://github.com/TyroneSlothrop/vassal-ts-emulator.git	Es el original. Solo se lee (fetch) para traerse novedades.

2.2 Comprobación del remote upstream

Para verificar que el remote upstream sigue configurado (se configuró una sola vez al inicio del proyecto):

```
cd %USERPROFILE%\vassal-ocs
git remote -v
```

Deben aparecer cuatro líneas, dos por cada remote (fetch y push):

```
origin    https://github.com/JaviARQ/vassal-ocs.git (fetch)
origin    https://github.com/JaviARQ/vassal-ocs.git (push)
upstream  https://github.com/TyroneSlothrop/vassal-ts-emulator.git
(fetch)
upstream  https://github.com/TyroneSlothrop/vassal-ts-emulator.git
(push)
```

Si upstream no aparece, se añade con:

```
git remote add upstream https://github.com/TyroneSlothrop/vassal-ts-
emulator.git
```

2.3 Rutina de sincronización

Antes de ejecutar la rutina, conviene echar un vistazo al historial del original para ver si hay cambios que merezcan la pena:

Visita `https://github.com/TyroneSlothrop/vassal-ts-emulator/commits/main` y mira los commits recientes. Si no hay nada nuevo desde la última sincronización, no hace falta hacer nada.

La rutina completa, ejecutar en cmd dentro del proyecto:

```
cd %USERPROFILE%\vassal-ocs

REM 1. Traerse los cambios del original (no modifica tus archivos)
git fetch upstream

REM 2. (Opcional) Ver qué commits hay nuevos que aún no tienes
git log HEAD..upstream/main --oneline

REM 3. Fusionar upstream/main con tu main
git merge upstream/main

REM 4. Si no hubo conflictos: empujar a tu repo
git push
```

Tras el git push, el workflow de GitHub Actions se dispara solo y reconstruye el sitio con la nueva versión del emulador.

> **Buena práctica: prueba en local antes de pushear**
>
> Si el merge trae muchos cambios (especialmente en código TypeScript del emulador), es recomendable probar antes en local con npm run dev y cargar Tunisia II para confirmar que nada se ha roto. Solo cuando funciona se hace el push.

2.4 Resolución de conflictos

Un conflicto ocurre cuando upstream modifica un archivo que tú también has modificado. En este proyecto, el único archivo sensible es scripts/gen-presets.mjs, donde aplicamos el parche Windows fileURLToPath.

Si al ejecutar git merge upstream/main aparece un mensaje como:

```
CONFLICT (content): Merge conflict in scripts/gen-presets.mjs
```

Git detiene el merge y marca el archivo con tres bloques delimitados por signos:

```
<<<<<<< HEAD
(tu versión local con fileURLToPath)
=======
(versión nueva que viene de upstream)
>>>>>>> upstream/main
```

Pasos para resolverlo:

- **Abrir el archivo** con el Bloc de notas o tu editor favorito.
- **Localizar los tres bloques** <<<<<<< HEAD, ======= y >>>>>>> upstream/main.
- **Decidir la versión final:** normalmente será mantener el fileURLToPath nuestro (que es un fix correcto) más cualquier mejora nueva del autor que sea compatible. Si el autor hubiera corregido él mismo el bug Windows, su versión ya sería buena y podríamos eliminar la nuestra.
- **Borrar las tres marcas** <<<<<<<, ======= y >>>>>>> y dejar el código final limpio.
- **Guardar el archivo** y en la cmd ejecutar:

```
git add scripts/gen-presets.mjs
git commit -m "merge upstream + mantener parche Windows"
git push
```

> **Abortar un merge si te pierdes**
>
> Si la resolución del conflicto se te complica, SIEMPRE puedes abortar y volver al estado anterior con: git merge --abort. Esto deja todo como estaba antes de empezar el merge, sin pérdida de datos.

2.5 Cuándo y cómo upstream nuestro parche Windows

El parche que aplicamos (sustituir .pathname por fileURLToPath) es un bug fix real que afecta a cualquier usuario de Windows, no algo específico de nuestro setup. Lo ideal sería enviarlo al autor original como Pull Request (PR). Si lo acepta:

- A partir de ese momento upstream ya lleva el parche.
- Nuestros futuros merges no tendrán conflicto en ese archivo.

- Otros usuarios en Windows se benefician.

Pasos para abrir el PR (operación opcional, hacer una sola vez):

```
REM En GitHub web: haz fork del original desde
REM https://github.com/TyroneSlothrop/vassal-ts-emulator
REM (botón Fork arriba a la derecha)

REM Añade tu fork como otro remote (lo llamamos "myfork")
git remote add myfork https://github.com/JaviARQ/vassal-ts-
emulator.git

REM Crea una rama específica para el PR
git checkout -b fix/windows-paths-gen-presets

REM Asegúrate de que el commit con el parche está en esa rama
REM (puedes cherry-pick el commit original o crear uno nuevo)

git push -u myfork fix/windows-paths-gen-presets
```

Tras ese push, ve a tu fork en GitHub (https://github.com/JaviARQ/vassal-ts-emulator) y aparecerá un banner verde con un botón "Compare & pull request". Sigue el asistente, escribe un título claro como "fix(gen-presets): use fileURLToPath for cross-platform paths" y una descripción explicando que el uso de .pathname rompe la generación de presets en Windows. El autor recibirá el PR y decidirá si lo mergea.

3. Añadir juegos nuevos al proyecto

Una vez desplegado el sitio, añadir un módulo nuevo es una operación rutinaria de seis pasos. Esta sección describe el procedimiento completo y, al final, una lista orientativa de módulos OCS con su estado de compatibilidad esperado.

3.1 Consideraciones previas

Antes de descargar cualquier .vmod conviene tener presentes tres cosas:

- **Clases Java personalizadas:** VASSAL permite que un módulo incluya código Java (.class) para automatizar reglas. El emulador TypeScript NO ejecuta Java. Módulos con Java dependiente pueden cargar (el formato se interpreta) pero sus botones automáticos no funcionarán o fallarán silenciosamente. Tunisia II desde la versión 1.0 está limpio de Java (reescrito sin el código de George Hayward), por eso carga bien. Otros OCS clásicos pueden tener Java.
- **Tamaño del archivo:** cada .vmod pesa entre 30 y 45 MB. GitHub recomienda repos menores de 1 GB y archivos menores de 100 MB. Con 8-10 módulos se está sobrado; con 25+ se acerca al límite y conviene considerar Git LFS.

- **Derechos de autor:** los módulos VASSAL de la biblioteca oficial (vassalengine.org/library) están publicados con permiso de los editores (MMP en el caso de OCS). Subirlos a un repo público propio como parte del emulador está en el mismo espíritu que el módulo original, pero sin redistribuir arte oficial del juego por separado.

3.2 Paso 1 — Descarga del .vmod

Los módulos OCS se descargan desde la biblioteca oficial de VASSAL:

Ve a `https://vassalengine.org/library` y busca el juego. Para la serie OCS puedes filtrar por la etiqueta `series: Operational Combat Series`.

Descarga siempre la versión más reciente (aparece la primera en la lista de Packages). El archivo .vmod se guarda en la carpeta Descargas del PC.

> **Aviso del navegador al descargar .vmod**
>
> Edge y Chrome pueden mostrar un aviso de "este tipo de archivo puede dañar tu equipo" porque la extensión .vmod no la reconocen. Es un ZIP perfectamente seguro. Pulsa Conservar / Mantener.

3.3 Paso 2 — Colocación en el proyecto

Cada módulo va en su propia carpeta bajo public/modules/. El nombre de la carpeta debe ser corto, en minúsculas y sin espacios (se usa como identificador en la URL final). Por ejemplo, para DAK:

```
mkdir %USERPROFILE%\vassal-ocs\public\modules\dak
copy "%USERPROFILE%\Downloads\DAK_x_y.vmod" "%USERPROFILE%\vassal-
ocs\public\modules\dak\"
```

Si el módulo trae extensiones (archivos .vmod adicionales que extienden el módulo base), se colocan en la misma carpeta:

```
public/modules/dak/
├── dak.vmod              ← módulo base
└── dak-extension.vmod    ← extensión (si existe)
```

3.4 Paso 3 — Generación del índice de presets

El script gen-presets.mjs escanea public/modules/ y genera (o actualiza) el archivo public/modules/presets.json que el emulador lee al arrancar para saber qué módulos hay disponibles.

```
cd %USERPROFILE%\vassal-ocs
npm run presets
```

Salida esperada (un "added" por cada módulo nuevo):

```
added dak: "OCS - DAK 2.0" (0 extensions)
wrote C:\Users\JaviPC\vassal-ocs\public\modules\presets.json (2
presets)
```

Si aparece un error ENOENT con ruta duplicada C:\C:\...

Significa que alguien ha modificado scripts/gen-presets.mjs y ha vuelto al código original (.pathname). Aplica de nuevo el parche descrito en la sección 2.4 (sustituir .pathname por fileURLToPath) y vuelve a ejecutar npm run presets.

3.5 Paso 4 — Prueba en local (dev server)

ANTES de pushear a GitHub, prueba el módulo nuevo en local para confirmar que carga bien:

```
cd %USERPROFILE%\vassal-ocs
npm run dev
```

Abre http://localhost:5173/ en el navegador, carga el módulo nuevo desde el selector y observa:

- ¿Aparece el mapa / tablero?
- ¿Aparecen las piezas en la paleta?
- ¿Puedes mover y seleccionar piezas sin errores en la consola (F12)?
- ¿Funcionan los menús y botones básicos?

Si la prueba va bien

Detén el dev server con Ctrl+C (S para confirmar). Ya puedes pasar al Paso 5.

Si el módulo no carga o falla algo

NO lo subas a GitHub todavía. Revisa la consola F12 para ver qué trait o pieza falla. Dependiendo del problema, la solución pasa por: (a) parchear el .vmod quitando el trait problemático (descomprimir, editar buildFile.xml, recomprimir); (b) añadir soporte para ese trait en el emulador TypeScript (src/); (c) aceptar que esa función no está disponible y jugar sin ella. Consulta el capítulo 6 Troubleshooting para diagnósticos concretos.

3.6 Paso 5 — Commit y push

Cuando el módulo funciona en local, se sube a GitHub:

```
cd %USERPROFILE%\vassal-ocs
git add -A
git status          ← verifica que solo aparecen los archivos del
módulo nuevo + presets.json
git commit -m "feat: añadir <nombre del módulo> <versión>"
git push
```

Verifica siempre el git status antes de committear

Tras git add -A, ejecuta git status. Si aparece cualquier cosa de node_modules/, aborta con git reset y revisa el .gitignore. El commit solo debe llevar: la carpeta nueva del módulo (public/modules/<nombre>/) y presets.json actualizado.

3.7 Paso 6 — Verificación del deploy

Tras el push, el workflow de GitHub Actions se dispara automáticamente. Monitorea el resultado en:

`https://github.com/JaviARQ/vassal-ocs/actions`

- 🟢 **Verde:** deploy OK, sitio actualizado en https://javiarq.github.io/vassal-ocs/ con el módulo nuevo disponible.
- 🔴 **Rojo:** mira el step que falla y el mensaje. Causas típicas: error de build TypeScript, módulo corrupto, espacio agotado en Pages.

Para que el módulo nuevo quede disponible offline en el iPad, hay que hacer una visita online cargando el módulo (ver capítulo 4).

3.8 Lista orientativa de módulos OCS

Esta tabla recoge los principales títulos de la serie OCS disponibles en la biblioteca de VASSAL. La columna "Java" indica el riesgo de encontrar clases Java personalizadas que el emulador no pueda ejecutar. Es orientativa; el estado real solo se confirma probando el módulo en local.

Tabla 1 — Módulos OCS en vassalengine.org/library

Módulo	OCS #	Tamaño aprox.	Riesgo Java	Notas
Tunisia II	#15	~39 MB	Bajo	Reescrito sin Java desde v1.0. Carga bien.
DAK (Desert Afrika Korps)	#1-2	~40 MB	Medio	Módulo clásico. Probar antes de pushear.
Guderian's Blitzkrieg II	#7	~45 MB	Medio	Rusia 1941. Muy grande.
Enemy at the Gates	#6	~42 MB	Medio	Stalingrado. Módulo extenso.
Hube's Pocket	#5	~35 MB	Medio	Ucrania 1944.
Sicily	#?	~38 MB	Medio	Operación Husky 1943.
Across the Piave	#?	~35 MB	Medio	Italia 1918 (WWI).

Módulo	OCS #	Tamaño aprox.	Riesgo Java	Notas
Korea: The Forgotten War	#?	~40 MB	Medio	Corea 1950.
Reluctant Enemies	#?	~38 MB	Bajo-Medio	Líbano/Siria 1941.
Crisis in the Ukraine	#?	~40 MB	Medio	Ucrania 1943-44.

Las versiones modernas (2020+) de muchos de estos módulos han sido reescritas por la comunidad eliminando parte del Java antiguo. Conviene siempre descargar la versión más reciente y comprobar la nota de release: si menciona "sin código de George Hayward" o "removed custom Java" el riesgo es bajo.

4. Actualización del iPad para juego offline

El servicio de cache offline se basa en un service worker (SW), un script JavaScript que el navegador instala la primera vez que visita el sitio y que a partir de entonces intercepta todas las peticiones de red: si hay conexión, sirve la versión más reciente y la cachea; si no hay, sirve la versión cacheada. Esto permite jugar a Tunisia II (o cualquier otro módulo cargado) en el iPad aunque no haya WiFi.

4.1 Concepto del service worker

El SW del proyecto (public/sw.js) usa dos estrategias según el tipo de recurso:

Tipo de recurso	Estrategia	Comportamiento
HTML, JS, CSS (interfaz)	Network-first	Siempre intenta red primero (versión nueva). Si no hay red, sirve cache.
.vmod, imágenes, fuentes	Stale-while-revalidate	Sirve cache al instante (rápido) y descarga la versión nueva en segundo plano para la próxima visita.

Además, el SW está configurado con self.skipWaiting() y self.clients.claim(), lo que significa que cuando se publica una versión nueva del SW, este se activa inmediatamente sin esperar a que se cierren todas las pestañas. Esto simplifica mucho la vida: no hace forzar cierres raros para que se actualice el sitio.

4.2 Procedimiento por módulo nuevo

Cuando añades un módulo nuevo al repo (capítulo 3) y el workflow se pone verde, el sitio online ya incluye ese módulo. PERO para que esté disponible OFFLINE en el iPad, hace falta que el iPad lo descargue al menos una vez. Pasos:

- **1. iPad online** (WiFi o datos).
- **2. Abre Safari** y ve a https://javiarq.github.io/vassal-ocs/.
- **3. Carga el módulo nuevo** desde el selector del emulador y espera a que el mapa y la paleta de piezas se vean completamente. Esto dispara la descarga del .vmod (30-45 MB) que el SW cacheará.
- **4. Verifica que el SW ha cacheado el archivo** abriendo en un PC (no en el iPad) el sitio y pulsando F12 → Application → Cache Storage. Debes ver una entrada con la URL del .vmod.
- **5. Prueba offline** poniendo el iPad en modo avión y abriendo la app desde el icono de pantalla de inicio. Carga el módulo: debe funcionar sin red.

> **Crítico: hay que CARGAR el módulo online al menos una vez**
>
> Si solo abres la home del sitio en el iPad y no cargas el módulo, el .vmod no se habrá descargado y por tanto no estará cacheado. Offline no funcionará para ese módulo. Siempre: visita online → carga el módulo concreto → espera a que esté visible del todo.

4.3 Verificación de caché

En el iPad no es fácil inspeccionar el SW. Lo más práctico es simplemente probar: poner en modo avión y abrir la app. Si carga, está cacheado. Si no carga y se queda en blanco, falta algo por cachear.

Para verificar en el PC (más visual):

- **Abre Chrome o Edge** en https://javiarq.github.io/vassal-ocs/.
- **Pulsa F12** para abrir DevTools.
- **Pestaña Application** → Service Workers. Debes ver "vassal-ocs-v1" con status "activated and is running".
- **Pestaña Cache Storage** → despliega "vassal-ocs-v1". Debes ver todas las entradas cacheadas: index.html, JS, CSS, fuentes y los .vmod que hayas cargado.

4.4 Cómo forzar refresco si la app no se actualiza

Si tras un deploy nuevo el iPad sigue mostrando la versión vieja, prueba en orden:

- **1. Cierra Safari del todo** (doble click home, desliza Safari hacia arriba). En iOS sin botón home: desliza desde abajo, desliza Safari hacia arriba.
- **2. Abre Safari, ve a la URL** y espera 5 segundos con la página visible. El SW nuevo (si lo hay) se descarga y se activa en segundo plano.
- **3. Recarga** deslizando hacia abajo desde la parte superior de la página (pull to refresh) o pulsando Cmd+R si tienes teclado.

- **4. Si sigue sin actualizarse:** ajustes de Safari → Safari → Avanzado → Datos de sitios web → busca javiarq.github.io y bórralo. Vuelve a visitar la URL online.

4.5 Cómo resetear la caché del SW por completo

Si la caché se ha corrompido o hay un .vmod a medias que impide cargar el módulo, se puede hacer un reset completo. Esto borra todo lo cacheado y obliga a una descarga limpia desde cero:

En el PC (Chrome/Edge)

```
F12 → Application → Service Workers
  → pulsa "Unregister" en el SW de vassal-ocs
F12 → Application → Storage
  → pulsa "Clear site data"
```

En el iPad

iOS no ofrece una interfaz directa para gestionar SW por sitio. El procedimiento es:

- **Ajustes → Safari → Avanzado → Datos de sitios web** → busca javiarq.github.io → Editar → Eliminar.
- **Ajustes → Safari → Borrar historial y datos de sitios web** (esto borra TODO, no solo este sitio; úsalo solo si lo anterior no basta).
- **Abre Safari** y vuelve a visitar la URL online. Se reinstala el SW desde cero.

> **Tras un reset, hay que recargar TODOS los módulos online**
>
> Si tenías tres módulos cacheados y haces un reset, los tres dejan de estar disponibles offline hasta que vuelvas a cargar cada uno online. Es tedioso pero es la única forma de garantizar una caché limpia.

5. Otras actividades de mantenimiento

Además de sincronizar con upstream y añadir módulos, hay varias tareas menores que conviene realizar de vez en cuando para mantener el proyecto sano.

5.1 Verificación del workflow GitHub Actions

Cada vez que se hace un push, el workflow se dispara. Es buena práctica abrir la pestaña Actions tras cada push y confirmar que se pone verde. Si se queda en rojo, hay que investigar inmediatamente antes de seguir añadiendo cambios (ver capítulo 6.3 para troubleshooting).

URL de monitorización: https://github.com/JaviARQ/vassal-ocs/actions

También se puede ver el historial: cada run se guarda con su commit asociado, duración y estado. Útil para detectar si un deploy empezó a fallar a partir de un commit concreto.

5.2 Renovación del Personal Access Token (PAT)

El PAT creado para autenticar los push tiene una cadencia (lo normal es 90 días). Cuando expira, los push fallan con un error 403 o "Authentication failed".

Comprobación previa: ve a https://github.com/settings/tokens y mira la columna "Expires". Si está en amarillo o rojo, renueva ya.

Renovación

- Ve a https://github.com/settings/tokens
- Localiza el token "vassal-ocs deploy".
- Click en "Regenerate token" (regenera con la misma cadencia) o elimínalo y crea uno nuevo.
- Copia el token nuevo.

Actualizar en Windows

Windows guarda las credenciales en el Gestor de credenciales. Hay dos formas de actualizarlas:

- **Opción A — Automática:** el primer push que falle te abrirá una ventana emergente de Git Credential Manager; vuelve a autorizar y listo.
- **Opción B — Manual:** abre "Administrador de credenciales" de Windows (Panel de control → Cuentas de usuario → Administrador de credenciales) → Credenciales de Windows → busca la entrada git:https://github.com → Eliminar. El próximo push te pedirá autenticarte de nuevo.

> **Configuración del PAT para que no caduque tan a menudo**
>
> Al crear el PAT, en "Expiration" puedes elegir "No expiration" (sin caducidad). GitHub muestra un aviso de seguridad porque es menos seguro, pero para un repo de hobby personal es razonable. Si lo prefieres más seguro, deja 90 días y renueva cuando toque.

5.3 Monitoreo del tamaño del repo

GitHub recomienda repos menores de 1 GB y archivos menores de 100 MB. Los .vmod pesan 30-45 MB cada uno, así que se está sobrado hasta 15-20 módulos. Para monitorizar:

Ve a https://github.com/JaviARQ/vassal-ocs/settings → mira el apartado "Repository size". O en cmd:

```
cd %USERPROFILE%\vassal-ocs
git count-objects -vH
```

Salida tipo:

```
count: 200
size: 80.00 MiB
in-pack: 0
packs: 0
size-pack: 0
prune-packable: 0
garbage: 0
size-garbage: 0
```

Si el repo se acerca a 1 GB, considerar:

- **Git LFS:** mueve los .vmod a almacenamiento LFS (no cuentan contra el límite del repo). Setup más complejo.
- **Módulos fuera del repo:** alojar los .vmod en otro sitio (Cloudflare R2, Backblaze B2) y que el emulador los cargue por URL. Requiere cambios en presets.json.

5.4 Bump del cache version (cuándo y por qué)

El service worker tiene una constante CACHE al principio del archivo public/sw.js. Actualmente vale "vassal-ocs-v1". Cuando se cambia el valor, todos los dispositivos que visiten el sitio harán una recarga completa (cache vieja borrada, nueva descargada).

Cuándo hacer bump:

- Si has hecho cambios sustanciales y quieres asegurar que todos los dispositivos ven la versión nueva sin depender de expiraciones de caché HTTP.
- Si sospechas que algún asset se ha cacheado mal y está causando errores.

Cómo hacerlo:

```
REM Abre public/sw.js con el Bloc de notas
REM Cambia la línea:
REM   const CACHE = 'vassal-ocs-v1';
REM por:
REM   const CACHE = 'vassal-ocs-v2';

REM Commit y push
git add public/sw.js
git commit -m "chore: bump service worker cache version"
git push
```

Tras un bump, TODOS los dispositivos tienen que recargar todo online

Si tienes varios dispositivos con módulos cacheados, después de un bump cada uno tiene que volver a visitar el sitio online Y recargar cada módulo concreto. Úsalo solo cuando sea necesario.

5.5 Backup del archivo presets.json

public/modules/presets.json es generado por npm run presets pero puede contener metadatos editados a mano (nombres, defaultBoards, pieceScale) según el README del proyecto original. Conviene hacer una copia de seguridad periódica:

```
copy %USERPROFILE%\vassal-ocs\public\modules\presets.json
%USERPROFILE%\Documents\vassal-ocs-presets-backup.json
```

OJO: este backup es solo local. Como el archivo ya está commiteado en git, la mejor copia de seguridad es el propio historial de GitHub. Si pierdes el archivo local, basta con git pull para recuperarlo.

5.6 Limpieza del .gitignore

Cada cierto tiempo, revisa el .gitignore y añade cualquier nuevo patrón de archivos que no deban subirse. Patrones típicos a verificar:

```
node_modules/
dist/
*.log
npm-debug.log*
.vscode/
.idea/
.DS_Store
Thumbs.db
*.local
*.bak
```

Si en algún momento el editor (VS Code, Notepad++) crea archivos temporales que se cuelan en git status, añade su patrón al .gitignore y vuelve a hacer git add -A.

5.7 Reportar bugs al autor original

Si al probar un módulo descubres un bug del emulador (no del módulo), es buena práctica reportarlo al autor original en https://github.com/TyroneSlothrop/vassal-ts-emulator/issues. Información útil para incluir:

- Módulo concreto y versión (p.ej. "Tunisia II 1.17.2").
- Qué acción intentabas (arrastrar pieza, abrir paleta, lanzar dado…).
- Mensaje de error exacto de la consola (F12 → Console).
- Screenshot del estado del tablero cuando falla.
- Tu navegador y sistema (Chrome 130 en Windows 11, Safari en iPadOS 17…).

El autor mantiene el proyecto activamente (últimos commits en julio 2026) y responde razonablemente rápido. Reportar bugs ayuda a mejorar el emulador para todos.

5.8 Probar nuevos módulos antes de subirlos

Ya se mencionó en el capítulo 3, pero conviene repetirlo como rutina: nunca subas un .vmod a GitHub sin haberlo probado antes en local con npm run dev. La prueba mínima consiste en:

- Cargar el módulo sin errores en la consola.
- Ver el mapa / tablero.
- Ver la paleta de piezas.
- Mover una pieza del palette al mapa.
- Seleccionar una pieza en el mapa.
- Probar un botón de menú (fase, dado, etc.).

Si todo esto funciona, el módulo está listo para subir. Si alguno falla, diagnostica antes de pushear.

6. Troubleshooting

Recopilación de errores típicos que pueden aparecer durante el mantenimiento del proyecto, con su diagnóstico y solución.

6.1 Errores de git

Síntoma	Causa probable	Solución
'git' no se reconoce como comando	Git no instalado o PATH no actualizado	Reinicia la ventana de cmd. Si persiste, reinstala Git for Windows marcando la opción de añadir al PATH.
fatal: not a git repository	Estás fuera de la carpeta del proyecto	Ejecuta cd %USERPROFILE%\vassal-osc antes de cualquier git.
error: failed to push some refs	Hay commits en remoto que no tienes localmente	Ejecuta git pull antes de git push.
Authentication failed / 403 al pushear	PAT caducado o incorrecto	Ve a https://github.com/settings/tokens y regenera el PAT. Ver sección 5.2.
CONFLICT (content): Merge conflict in X	Merge con cambios simultáneos en mismo archivo	Resuelve manualmente (ver sección 2.4) o aborta con git merge --abort.
Updates were rejected because the remote contains work	Repo remoto con commits que no tienes	git pull --rebase origin main y luego git push.
warning: LF will be replaced by CRLF	Solo un aviso (line endings)	Ignóralo. No es un error.

Síntoma	Causa probable	Solución
'cgit' no se reconoce...	Typo al escribir 'git'	Escribe el comando correctamente: git ...

6.2 Errores de npm

Síntoma	Causa probable	Solución
'npm' no se reconoce como comando	Node no instalado o PATH no actualizado	Reinicia cmd. Si persiste, reinstala Node.js LTS.
npm ERR! ENOENT durante npm install	Alguna dependencia no se descarga bien	Borra node_modules y package-lock.json y vuelve a npm install.
npm WARN deprecated	Solo avisos, no errores	Ignóralos. Mientras no aparezca npm ERR! al final, está bien.
npm WARN install-scripts (esbuild)	Aviso sobre postinstall de esbuild	Normalmente es solo informativo. Si el dev server falla, ejecuta npm install-scripts approve esbuild y npm rebuild esbuild.
Error: ENOENT: no such file or directory 'C:\C:\...'	Bug del gen-presets.mjs sin parchear	Aplica el parche descrito en sección 2.4: sustituir .pathname por fileURLToPath en scripts/gen-presets.mjs.
Cannot find module 'docx' (u otro)	Dependencia no instalada	Ejecuta npm install en la raíz del proyecto.
Vite no arranca: EADDRINUSE :::5173	Puerto 5173 ya ocupado por otro proceso	Cierra el otro proceso o arranca con npx vite --port 5174.

6.3 Errores del workflow GitHub Actions

Step que falla	Causa probable	Solución
actions/configure-pages → Not Found	Pages no activado con source "GitHub Actions"	Settings → Pages → Source: GitHub Actions. Re-run del workflow.
npm ci → error	package-lock.json corrupto o ausente	En local: rm package-lock.json && npm install. Commit del nuevo lock y push.
npm run presets → ENOENT C:\C:\	El workflow corre en Linux; no debería pasar esto	Si pasara (no debería), el parche Windows está mal aplicado. Revisa scripts/gen-presets.mjs.

Step que falla	Causa probable	Solución
npm run build → TypeScript error	Error en código TS del emulador	Mira el log del step Build. Prueba en local con npm run build para reproducir.
upload-pages-artifact → size limit	El dist generado pesa > 10 GB (límite de Pages)	Impensable con un solo módulo. Solo pasará con docenas de .vmod.
deploy-pages → HttpError	Pages mal configurado o timeout	Re-run del workflow. Si persiste, mira Settings → Pages.

6.4 Errores de carga del módulo en el emulador

Síntoma en navegador	Causa probable	Solución
Pantalla en blanco al cargar módulo	Error JS en consola (F12)	Abre F12 → Console y mira el error. Suele ser un trait no soportado o una pieza corrupta.
Error 'Unknown trait' en consola	El emulador no implementa ese trait	Anótalo. Puedes (a) reportar bug al autor, (b) añadir soporte en src/, (c) editar buildFile.xml del .vmod para quitar el trait.
No se ven las piezas en la paleta	Problema con la decodificación de pieces o prototypes	F12 → Console. Suele venir con stack trace del parser.
Mapa se ve pero hex grid desalineado	Bug en el rendering de grid	Reporta issue al autor con screenshot.
Botón de menú no responde al click	Probablemente dependa de clase Java	Sin Java no se puede ejecutar. Juega la función manualmente.
.vmod tarda una eternidad en cargar	Módulo grande (≥ 45 MB) o conexión lenta	Normal. La primera carga cachea; las siguientes van por SW.

6.5 Errores en el iPad

Síntoma en iPad	Causa probable	Solución
Sitio no carga con WiFi	Problema de red o DNS	Prueba otra red. Verifica que https://javiarq.github.io/vassal-ocs/ carga en otro dispositivo.
Sitio carga pero módulo no abre offline	Módulo no cacheado previamente	Conecta a WiFi, abre el sitio, carga el módulo concreto (ver sección 4.2).

Síntoma en iPad	Causa probable	Solución
App abre pero se ve versión vieja	SW cacheando versión antigua	Ver sección 4.4 (cómo forzar refresco).
Safari se cierra al cargar módulo grande	Memoria agotada en iPad antiguo	Cierra otras apps. Considera cargar el módulo en un iPad con más RAM.
No aparece el icono de "Añadir a pantalla de inicio"	Opción de Safari mal buscada	Botón Compartir (cuadrado con flecha arriba) → scroll → "Añadir a pantalla de inicio".
Tapping icono abre Safari en vez de pantalla completa	Modo PWA no activado	Asegúrate de añadir desde Safari (no desde Chrome) y de usar "Añadir a pantalla de inicio", no "Añadir a marcadores".
Errores de certificado al abrir el sitio	Reloj del iPad mal puesto	Ajustes → General → Fecha y hora → Activa "Ajustar automáticamente".

7. Cheat sheet — comandos rápidos

Tabla de referencia con los comandos más usados en el mantenimiento del proyecto. Imprimible o consultable de un vistazo.

7.1 Setup inicial (una sola vez)

Tabla 2 — Comandos de setup inicial

Acción	Comando
Entrar al proyecto	cd %USERPROFILE%\vassal-ocs
Añadir remote upstream	git remote add upstream https://github.com/TyroneSlothrop/vassal-ts-emulator.git
Configurar identidad git	git config --global user.name "Tu Nombre"
Configurar email git	git config --global user.email tu@email.com

7.2 Rutinas habituales

Tabla 3 — Comandos de mantenimiento

Acción	Comando
Sincronizar con upstream	git fetch upstream && git merge upstream/main && git push

Acción	Comando
Añadir módulo nuevo	npm run presets && npm run dev (probar) && git add -A && git commit && git push
Generar presets	npm run presets
Arrancar dev server	npm run dev
Build producción local	npm run build
Ver status de git	git status
Ver qué ha cambiado	git diff
Ver remotes	git remote -v
Ver log reciente	git log --oneline -10
Deshacer git add	git reset
Deshacer git add de un archivo	git reset <archivo>
Abortar un merge en conflicto	git merge --abort

7.3 URLs importantes

Tabla 4 — URLs de referencia

Concepto	URL
Repositorio GitHub	https://github.com/JaviARQ/vassal-ocs
Sitio Pages (jugar)	https://javiarq.github.io/vassal-ocs/
Workflow Actions	https://github.com/JaviARQ/vassal-ocs/actions
Configuración Pages	https://github.com/JaviARQ/vassal-ocs/settings/pages
Repositorio original (upstream)	https://github.com/TyroneSlothrop/vassal-ts-emulator
Biblioteca de módulos VASSAL	https://vassalengine.org/library
Página del módulo Tunisia II	https://vassalengine.org/library/projects/Tunisia_II
Personal Access Tokens	https://github.com/settings/tokens

7.4 Archivos clave del proyecto

Tabla 5 — Archivos importantes y su función

Archivo	Función
package.json	Dependencias y scripts npm (presets, dev, build)

Archivo	Función
vite.config.ts	Configuración del bundler. base: "./" para subdirectorio
.github/workflows/deploy.yml	Workflow de GitHub Actions: build + deploy a Pages
public/sw.js	Service worker (offline cache)
public/sw-register.js	Registro del SW desde index.html
index.html	Entrada del emulador; carga el SW
scripts/gen-presets.mjs	Genera presets.json (PARCHEADO para Windows)
public/modules/presets.json	Índice de módulos disponibles
public/modules/<modulo>/<modulo>.vmod	Cada módulo VASSAL descargado
.gitignore	Excluye node_modules, dist, etc.

8. Glosario

Definiciones de los términos técnicos que aparecen en este documento y en el día a día del proyecto.

8.1 Git y GitHub

Término	Definición
git	Sistema de control de versiones distribuido. Registra cambios en archivos y permite trabajar con varios repositorios remotos.
commit	Muestra un conjunto de cambios registrados en git con un mensaje descriptivo.
push	Envía tus commits locales al repositorio remoto (origin o el que sea).
pull	Trae commits del repositorio remoto a tu copia local.
fetch	Como pull pero sin fusionar. Solo descarga los cambios; tú decides cuándo aplicarlos.
merge	Fusiona dos conjuntos de cambios en uno. Se usa para traer cambios de upstream a tu main.
rebase	Alternativa a merge que reescribe el historial local. Más limpio pero más peligroso (requiere force push).
clone	Crea una copia local completa de un repositorio remoto.

Término	Definición
fork	Copia de un repo en tu cuenta de GitHub. Se hace desde el botón Fork en la web.
remote	Puntero a un repositorio remoto. Por defecto al clonar se crea "origin".
origin	Remote apuntando a TU repo (github.com/JaviARQ/vassal-ocs).
upstream	Remote apuntando al repo original (github.com/TyroneSlothrop/vassal-ts-emulator).
PAT (Personal Access Token)	Contraseña larga generada en GitHub para autenticar push desde la línea de comandos. Sustituye a la contraseña normal desde 2021.
GCM (Git Credential Manager)	Herramienta incluida en Git for Windows que gestiona autenticación vía navegador OAuth.
main	Nombre de la rama principal del proyecto. Antes se llamaba "master".
HEAD	Puntero al commit en el que estás actualmente.
stash	Guardar cambios sin commitear temporalmente, para recuperarlos después.
diff	Muestra las diferencias entre dos versiones de archivos.

8.2 Node.js, npm y Vite

Término	Definición
Node.js	Entorno de ejecución de JavaScript fuera del navegador. Necesario para construir el proyecto.
npm	Gestor de paquetes de Node. Instala dependencias (npm install) y ejecuta scripts (npm run X).
package.json	Archivo que lista dependencias y scripts del proyecto.
package-lock.json	Snapshot de las versiones exactas instaladas. Garantiza builds reproducibles.
node_modules/	Carpeta con todas las dependencias instaladas. No se sube a git (reconstruida con npm install).
npm ci	Instalación limpia basada en package-lock.json. Más rápida y reproducible que npm install. La usa el workflow de Actions.
Vite	Bundler moderno para proyectos web. Compila TypeScript y genera dist/ optimizado para producción.
dev server	Servidor local de desarrollo (npm run dev). Sirve el sitio en http://localhost:5173.
dist/	Carpeta con el build de producción. Generada por npm run build, no se sube a git.

Término	Definición
TypeScript (TS)	Superset de JavaScript con tipos estáticos. El emulador está escrito en TS.

8.3 VASSAL y OCS

Término	Definición
VASSAL	Engine open-source para jugar juegos de tablero y cartas online. Originalmente en Java.
.vmod	Archivo de módulo VASSAL. Es un ZIP renombrado que contiene buildFile.xml, moduledata y recursos.
buildFile.xml	XML dentro del .vmod que define piezas, tableros, menús, traits. Lo lee el emulador TS.
moduledata	Archivo dentro del .vmod con metadatos: nombre, versión, autor.
trait	Comportamiento de una pieza (rotación, etiqueta, marca de suministro, etc.). Cada pieza puede tener varios traits apilados.
Java personalizado	Clases .class dentro de un .vmod que implementan lógica avanzada. El emulador TS no las ejecuta.
George Hayward	Colaborador histórico de módulos OCS que añadió mucho Java personalizado. Tunisia II desde v1.0 está limpio de su código.
OCS	Operational Combat Series. Serie de wargames operacionales publicada por MMP (Multi-Man Publishing).
MMP	Multi-Man Publishing. Editorial que publica OCS y otros wargames.
preset	Entrada en presets.json que describe un módulo cargable: nombre, archivos, extensiones.
.vsav	Archivo de partida guardada VASSAL. Compatible entre el emulador TS y VASSAL Java de escritorio.

8.4 Service worker y PWA

Término	Definición
Service worker (SW)	Script JavaScript que el navegador instala en segundo plano y que intercepta peticiones de red para gestionar caché offline.
PWA (Progressive Web App)	Aplicación web que, con un SW y un manifest, se comporta como una app nativa: icono en pantalla de inicio, pantalla completa, offline.
cache	Almacenamiento local de assets (HTML, JS, .vmod). El SW lo gestiona.

Término	Definición
cache version	Etiqueta única ("vassal-ocs-v1") en el SW. Al cambiarla, se invalida la caché vieja y se descarga todo de nuevo.
network-first	Estrategia: probar red primero (versión nueva); si falla, servir cache.
stale-while-revalidate	Estrategia: servir cache al instante y descargar versión nueva en segundo plano para la próxima visita.
skipWaiting()	Método del SW que activa la nueva versión inmediatamente sin esperar a que se cierren las pestañas.
clients.claim()	Método del SW que toma control de las pestañas abiertas inmediatamente al activarse.

8.5 GitHub Pages y GitHub Actions

Término	Definición
GitHub Pages	Servicio gratuito de hosting estático integrado en GitHub. Sirve sitios web desde repos.
GitHub Actions	Sistema de CI/CD integrado en GitHub. Ejecuta workflows (workflows YAML) en respuesta a eventos (push, PR, etc.).
workflow	Archivo YAML en .github/workflows/ que define un job: pasos, entorno, permisos.
runner	Servidor donde se ejecuta el workflow (ubuntu-latest, windows-latest, etc.).
step	Cada paso dentro de un job: checkout, setup-node, npm ci, etc.
artifact	Colección de archivos producida por un workflow y subida a GitHub para uso posterior (p.ej. dist/ que se deploya a Pages).
Source: GitHub Actions	Configuración de Pages que indica que el sitio se publica desde un workflow, no desde una rama.